

EPIC 05 — Task 03: Circuit Breaker

The task requires implementing:

- **Closed state**
- **Open state**
- **Half-open state**
- **Fallback response**

We will use **Resilience4j Circuit Breaker** with the `ProductClient` from Task 01/02.

Step 1 — Add Circuit Breaker dependency

Open your **Order Service** → `pom.xml`.

Add:

```
<dependency>
  <groupId>io.github.resilience4j</groupId>
  <artifactId>resilience4j-spring-boot3</artifactId>
</dependency>
```

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-aop</artifactId>
</dependency>
```

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

Save the file and let Maven download the dependencies.

Step 2 — Configure the Circuit Breaker

Open:

src/main/resources/application.properties

Add:

resilience4j.circuitbreaker.instances.productService.register-health-indicator=true

resilience4j.circuitbreaker.instances.productService.sliding-window-type=COUNT_BASED

resilience4j.circuitbreaker.instances.productService.sliding-window-size=5

resilience4j.circuitbreaker.instances.productService.minimum-number-of-calls=5

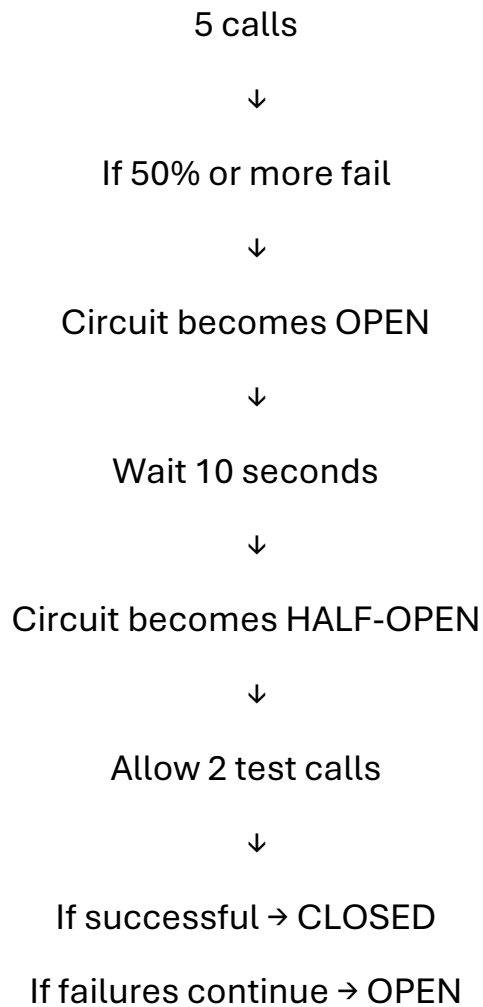
resilience4j.circuitbreaker.instances.productService.failure-rate-threshold=50

resilience4j.circuitbreaker.instances.productService.wait-duration-in-open-state=10s

resilience4j.circuitbreaker.instances.productService.permitted-number-of-calls-in-half-open-state=2

resilience4j.circuitbreaker.instances.productService.automatic-transition-from-open-to-half-open-enabled=true

These settings mean,



Step 3 — Add Circuit Breaker to ProductClient

Open:

`ProductClient.java`

Use:

```
package com.example.orderservice.client;
```

```
import org.springframework.stereotype.Component;
```

```
import org.springframework.web.client.RestClient;
```

```
import io.github.resilience4j.circuitbreaker.annotation.CircuitBreaker;
import io.github.resilience4j.retry.annotation.Retry;
```

```
@Component
```

```
public class ProductClient {
```

```
    private final RestClient restClient;
```

```
    public ProductClient(RestClient restClient) {
        this.restClient = restClient;
    }
```

```
    @CircuitBreaker(
        name = "productService",
        fallbackMethod = "getProductFallback"
    )
```

```
    @Retry(name = "productService")
```

```
    public ProductResponse getProductById(Long productId) {
```

```
        return restClient.get()
            .uri("/products/{id}", productId)
            .retrieve()
            .body(ProductResponse.class);
    }
```

```
    public ProductResponse getProductFallback(
        Long productId,
        Exception exception) {
```

```
        ProductResponse response = new ProductResponse();
```

```
        response.setId(productId);
```

```
        response.setName("Product service unavailable");
```

```
        response.setPrice(0.0);

        return response;
    }
}
```

The important part is:

```
@CircuitBreaker(
    name = "productService",
    fallbackMethod = "getProductFallback"
)
```

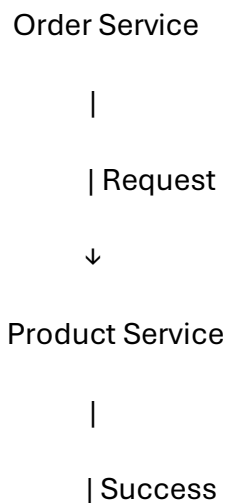
This tells Resilience4j to protect the Product Service call.

Step 4 — Understand the Closed State

Initially, the circuit is:

CLOSED

The request goes normally to Product Service.





Order Service

Failures are counted while the circuit is **CLOSED**.

For our configuration, after at least **5 calls**, if the failure rate reaches **50%**, the circuit opens.

Step 5 — Understand the Open State

If Product Service keeps failing:

CLOSED



Too many failures



OPEN

When the circuit is **OPEN**, calls are stopped immediately.

Order Service



X

Circuit OPEN



Fallback Response

This prevents the Order Service from continuously calling an unavailable Product Service.

Step 6 — Understand the Half-Open State

After:

wait-duration-in-open-state = 10s

the circuit changes to:

HALF-OPEN

It allows:

2 test calls

because we configured:

permitted-number-of-calls-in-half-open-state=2

If the calls succeed:

HALF-OPEN



Success



CLOSED

If they fail:

HALF-OPEN



Failure



OPEN

Step 7 — Fallback Response

If Product Service is unavailable, this method is executed:

```
public ProductResponse getProductFallback(  
    Long productId,  
    Exception exception) {  
  
    ProductResponse response = new ProductResponse();  
  
    response.setId(productId);  
    response.setName("Product service unavailable");  
    response.setPrice(0.0);  
  
    return response;  
}
```

Instead of returning an error to the user, the Order Service can return:

```
{  
  
    "id": 1,  
    "name": "Product service unavailable",  
    "price": 0.0  
}
```


Step 8 — Test the Circuit Breaker

Start both services:

Order Service → 8081

Product Service → 8082

Test:

GET <http://localhost:8081/orders/product/1>

Normal condition

Order Service



Circuit CLOSED



Product Service



Product Response

Stop Product Service

Make several requests.

You will eventually get:

Order Service



Circuit OPEN



Fallback



"Product service unavailable"

Wait approximately **10 seconds**.

The circuit enters:

HALF-OPEN

It allows the configured test calls.

If Product Service is running again:

HALF-OPEN

↓

SUCCESS

↓

CLOSED

Final project structure

Order Service

|

├─ config

|

└─ RestClientConfig.java

|

├─ client

|

├─ ProductClient.java

|

└─ ProductResponse.java

|

└─ service

```

|      └─ OrderService.java
|
|
└─ controller
    |      └─ OrderController.java
    |
    └─ resources
        └─ application.properties

```

Requirement

Closed state
 Open state
 Half-open state
 Fallback response
 Circuit Breaker
 Failure threshold
 Sliding window

Implementation

Normal requests pass through
 Calls are blocked after failure threshold
 2 test calls after 10 seconds
 getProductFallback()
 @CircuitBreaker
 50%
 5 calls

Task 03- Circuit Breaker is complete.

